МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-13

Выполнил:

А.М. Литвинчук

студент группы ПО-13

Проверил:

А. Д. Кулик

13.02.2026

Брест 2026

Лабораторная работа 6

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)

1. Создайте файл test_cart.py. Реализуйте следующие тесты:

• Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.

• Проверка выброса ошибки при отрицательной цене.

• Проверка вычисления общей стоимости (total()).

2. Протестируйте метод apply_discount с разными значениями скидки:

• 0% - цена остаётся прежней

• 50% - цена уменьшается вдвое

• 100% - цена становится ноль

• < 0% и > 100% - должно выбрасываться исключение

Используйте @pytest.mark.parametrize

3. Создайте фикстуру empty_cart, которая возвращает пустой экземпляр Cart

```
@pytest.fixture
def empty_cart():
return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
def log_purchase(item):
requests.post("https://example.com/log", json=item)
```

• Замокайте requests.post, чтобы не было реального HTTP-запроса

• Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):
coupons = {"SAVE10": 10, "HALF": 50}
if coupon_code in coupons:
cart.apply_discount(coupons[coupon_code])
else:
raise ValueError("Invalid coupon")
```

• Напишите тесты на apply_coupon

• Замокайте словарь coupons с помощью monkeypatch или patch.dict

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Вариант:3

3) Напишите метод String keep(String str, String pattern) который оставляет

в первой строке все символы, которые присутствуют во второй.

Спецификация метода:

keep (None , None) = TypeError

keep (None, *) = None

keep ("", *) = ""

keep (* , None) = ""

keep (* , "") = ""

keep (" hello ", "hl") = " hll "

keep (" hello ", "le") = " ell "

```
def rep(start_value: int, end_value: int, step_value: int) -> list[int]:
    if step_value == 0:
        raise ValueError("Step cannot be zero")

    return list(range(start_value, end_value + 1, step_value))


def is_palindrome(text: str) -> bool:
    filtered = "".join(char for char in text.lower() if char.isalnum())
    return filtered == filtered[::-1]
import requests

COUPONS = {
    "SAVE10": 10,
    "HALF": 50,
}


class Cart:
    def __init__(self):
        self.items = []
        self.discount = 0

    def add_item(self, name, price):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append({"name": name, "price": price})

    def total(self):


        total_price = sum(item["price"] for item in self.items)

        return total_price * (1 - self.discount / 100)

    def apply_discount(self, percent):
        if percent < 0 or percent > 100:
            raise ValueError("Discount must be between 0 and 100")
        self.discount = percent


def log_purchase(item):
    requests.post("https://example.com/log", json=item)
```

```python
def apply_coupon(cart, coupon_code):
    if coupon_code in COUPONS:
        cart.apply_discount(COUPONS[coupon_code])
    else:
        raise ValueError("Invalid coupon")
def keep(string, pattern):
    if string is None and pattern is None:
        raise TypeError("Both arguments are None")

    if string is None:
        return None

    if string == "":
        return ""

    if pattern is None or pattern == "":
        return ""

    result = ""

    for ch in string:
        if ch in pattern or ch == " ":
            result += ch

    return result
import pytest
from shopping import Cart, log_purchase, apply_coupon


@pytest.fixture
def empty_cart():
    return Cart()


def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)

    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0


def test_add_item_negative_price(empty_cart):
    with pytest.raises(ValueError, match="Price cannot be negative"):
        empty_cart.add_item("Apple", -10.0)


def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)

    assert empty_cart.total() == 15.0


@pytest.mark.parametrize(
```

```python
    "discount, expected_total",
    [
        (0, 100.0),
        (50, 50.0),
        (100, 0.0),
    ],
)
def test_apply_discount_valid(empty_cart, discount, expected_total):
    empty_cart.add_item("Item", 100.0)
    empty_cart.apply_discount(discount)

    assert empty_cart.total() == expected_total


@pytest.mark.parametrize("discount", [-1, 101])
def test_apply_discount_invalid(empty_cart, discount):
    with pytest.raises(ValueError, match="Discount must be between 0 and 100"):
        empty_cart.apply_discount(discount)


from unittest.mock import patch
import pytest
from shopping import Cart, log_purchase, apply_coupon


@pytest.fixture
def empty_cart():
    return Cart()


def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)

    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0


def test_add_item_negative_price(empty_cart):
    with pytest.raises(ValueError, match="Price cannot be negative"):
        empty_cart.add_item("Apple", -10.0)


def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)

    assert empty_cart.total() == 15.0


@pytest.mark.parametrize(
    "discount, expected_total",
    [
        (0, 100.0),
        (50, 50.0),
        (100, 0.0),
```

```python
    ],
)
def test_apply_discount_valid(empty_cart, discount, expected_total):
    empty_cart.add_item("Item", 100.0)
    empty_cart.apply_discount(discount)

    assert empty_cart.total() == expected_total


@pytest.mark.parametrize("discount", [-1, 101])
def test_apply_discount_invalid(empty_cart, discount):
    with pytest.raises(ValueError, match="Discount must be between 0 and 100"):
        empty_cart.apply_discount(discount)


def test_log_purchase_calls_requests_post():
    item = {"name": "Apple", "price": 10.0}

    with patch("shopping.requests.post") as mocked_post:
        log_purchase(item)
        mocked_post.assert_called_once_with(
            "https://example.com/log",
            json=item,
        )


def test_apply_coupon_save10(empty_cart):
    empty_cart.add_item("Item", 100.0)

    apply_coupon(empty_cart, "SAVE10")

    assert empty_cart.total() == 90.0


def test_apply_coupon_half(empty_cart):
    empty_cart.add_item("Item", 100.0)

    apply_coupon(empty_cart, "HALF")

    assert empty_cart.total() == 50.0


def test_apply_coupon_invalid(empty_cart):
    empty_cart.add_item("Item", 100.0)

    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(empty_cart, "WRONG")


def test_apply_coupon_with_monkeypatch(empty_cart, monkeypatch):
    from shopping import COUPONS

    empty_cart.add_item("Item", 200.0)
    monkeypatch.setitem(COUPONS, "SUPER", 25)
```

```python
    apply_coupon(empty_cart, "SUPER")

    assert empty_cart.total() == 150.0


def test_apply_coupon_save10(empty_cart):
    empty_cart.add_item("Item", 100.0)

    apply_coupon(empty_cart, "SAVE10")

    assert empty_cart.total() == 90.0


def test_apply_coupon_half(empty_cart):
    empty_cart.add_item("Item", 100.0)

    apply_coupon(empty_cart, "HALF")

    assert empty_cart.total() == 50.0


def test_apply_coupon_invalid(empty_cart):
    empty_cart.add_item("Item", 100.0)

    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(empty_cart, "WRONG")


def test_apply_coupon_with_monkeypatch(empty_cart, monkeypatch):
    from shopping import COUPONS

    empty_cart.add_item("Item", 200.0)
    monkeypatch.setitem(COUPONS, "SUPER", 25)

    apply_coupon(empty_cart, "SUPER")

    assert empty_cart.total() == 150.0
import pytest
from lab1_utils import rep, is_palindrome


def test_rep_normal_case():
    assert rep(1, 5, 1) == [1, 2, 3, 4, 5]


def test_rep_with_step_two():
    assert rep(1, 5, 2) == [1, 3, 5]


def test_rep_single_value():
    assert rep(3, 3, 1) == [3]


def test_rep_empty_when_start_greater_than_end():
    assert rep(5, 1, 1) == []
```

```python
def test_rep_zero_step_raises():
    with pytest.raises(ValueError, match="Step cannot be zero"):
        rep(1, 5, 0)


def test_is_palindrome_simple_true():
    assert is_palindrome("abba") is True


def test_is_palindrome_simple_false():
    assert is_palindrome("hello") is False


def test_is_palindrome_mixed_case():
    assert is_palindrome("AbBa") is True


def test_is_palindrome_with_spaces_and_punctuation():
    assert is_palindrome("A man, a plan, a canal: Panama!") is True


def test_is_palindrome_empty_string():
    assert is_palindrome("") is True


def test_is_palindrome_digits():
    assert is_palindrome("12321") is True


def test_is_palindrome_alphanumeric_false():
    assert is_palindrome("abc123") is False
import pytest
from string_utils import keep


def test_keep_none_none():
    with pytest.raises(TypeError):
        keep(None, None)


def test_keep_none_pattern():
    assert keep(None, "abc") is None


def test_keep_empty_string():
    assert keep("", "abc") == ""


def test_keep_pattern_none():
    assert keep("hello", None) == ""


def test_keep_pattern_empty():
    assert keep("hello", "") == ""


def test_keep_hl():
```

```python
    assert keep(" hello ", "hl") == " hll "


def test_keep_le():
    assert keep(" hello ", "le") == " ell "


def test_keep_no_matches():
    assert keep("abc", "xyz") == ""


def test_keep_all_matches():
    assert keep("abc", "abc") == "abc"


def test_keep_repeated_chars():
    assert keep("hello world", "lo") == "llo ol"
```

```
PS C:\Users\Legion\Documents\GitHub\spp_po13\reports\Litvinchuk\6> python -m pytest -v
collected 35 items

src/test_cart.py::test_add_item PASSED                                                    [  2%]
src/test_cart.py::test_add_item_negative_price PASSED                                     [  5%]
src/test_cart.py::test_total PASSED                                                       [  8%]
src/test_cart.py::test_apply_discount_valid[0-100.0] PASSED                               [ 11%]
src/test_cart.py::test_apply_discount_valid[50-50.0] PASSED                               [ 14%]
src/test_cart.py::test_apply_discount_valid[100-0.0] PASSED                               [ 17%]
src/test_cart.py::test_apply_discount_invalid[-1] PASSED                                  [ 20%]
src/test_cart.py::test_apply_discount_invalid[101] PASSED                                 [ 22%]
src/test_cart.py::test_log_purchase_calls_requests_post PASSED                            [ 25%]
src/test_cart.py::test_apply_coupon_save10 PASSED                                         [ 28%]
src/test_cart.py::test_apply_coupon_half PASSED                                           [ 31%]
src/test_cart.py::test_apply_coupon_invalid PASSED                                        [ 34%]
src/test_cart.py::test_apply_coupon_with_monkeypatch PASSED                               [ 37%]
src/test_lab1_utils.py::test_rep_normal_case PASSED                                       [ 40%]
src/test_lab1_utils.py::test_rep_with_step_two PASSED                                     [ 42%]
src/test_lab1_utils.py::test_rep_single_value PASSED                                      [ 45%]
src/test_lab1_utils.py::test_rep_empty_when_start_greater_than_end PASSED                 [ 48%]
src/test_lab1_utils.py::test_rep_zero_step_raises PASSED                                  [ 51%]
src/test_lab1_utils.py::test_is_palindrome_simple_true PASSED                             [ 54%]
src/test_lab1_utils.py::test_is_palindrome_simple_false PASSED                            [ 57%]
src/test_lab1_utils.py::test_is_palindrome_mixed_case PASSED                              [ 60%]
src/test_lab1_utils.py::test_is_palindrome_with_spaces_and_punctuation PASSED             [ 62%]
src/test_lab1_utils.py::test_is_palindrome_empty_string PASSED                            [ 65%]
src/test_lab1_utils.py::test_is_palindrome_digits PASSED                                  [ 68%]
src/test_lab1_utils.py::test_is_palindrome_alphanumeric_false PASSED                      [ 71%]
src/test_string_utils.py::test_keep_none_none PASSED                                      [ 74%]
src/test_string_utils.py::test_keep_none_pattern PASSED                                   [ 77%]
src/test_string_utils.py::test_keep_empty_string PASSED                                   [ 80%]
src/test_string_utils.py::test_keep_pattern_none PASSED                                   [ 82%]
src/test_string_utils.py::test_keep_pattern_empty PASSED                                  [ 85%]
src/test_string_utils.py::test_keep_hl PASSED                                             [ 88%]
src/test_string_utils.py::test_keep_le PASSED                                             [ 91%]
src/test_string_utils.py::test_keep_no_matches PASSED                                     [ 94%]
src/test_string_utils.py::test_keep_all_matches PASSED                                    [ 97%]
src/test_string_utils.py::test_keep_repeated_chars PASSED                                 [100%]

============================================================================ 35 passed in 0.20s ============================================================================
PS C:\Users\Legion\Documents\GitHub\spp_po13\reports\Litvinchuk\6>
```

Вывод: В ходе лабораторной работы были написаны и успешно выполнены тесты с использованием `pytest`, освоены фикстуры, параметризация и мокирование. Также были протестированы функции из предыдущей лабораторной работы и реализован метод `keep` по заданной спецификации. Все тесты прошли успешно: `35 passed`.